

FRONTEND TECHNICAL DOCUMENTATION

Digital Subsidy and Grant Administration Platform

React 19 · Vite · React Router 7 · Axios

Prepared for: Development, Evaluation & Maintenance Reference
August 2026

Table of Contents

1. Project Overview
2. Frontend Technology Stack
3. Frontend Architecture
4. Frontend Folder Structure
5. Application Flow
6. Routing
7. Components
8. Pages / Screens
9. State Management
10. API Integration
11. Authentication & Authorization
12. Forms & Validation
13. UI / Styling Architecture
14. Environment Configuration
15. Frontend Setup
16. Frontend Data Flow Examples
17. Error Handling
18. Build & Deployment
19. Frontend Maintenance Guide
20. Frontend Troubleshooting

1. Project Overview

The Digital Subsidy and Grant Administration Platform is a full-stack web application for managing the end-to-end lifecycle of government subsidy programs — from beneficiary application submission through officer verification, admin approval, officer allocation, field inspection, and milestone-based fund disbursement.

Purpose of the Frontend

The frontend is a single-page application (SPA) that serves as the interface layer for four distinct user roles — Beneficiary, Officer, Finance Officer, and Admin — each with a separate dashboard and workflow. It handles form-based data entry, real-time status tracking, document and media uploads, and analytics visualization, while delegating all business logic, persistence, and authentication decisions to the backend.

Role of the Frontend in the Overall System

The frontend communicates exclusively with the Spring Boot backend via a REST API (JSON) over HTTP, using a single shared Axios client. Authentication state is maintained via an HttpOnly cookie issued by the backend; the frontend does not independently manage or store a JWT token.

Main Technologies / Frameworks Used

- React 19 (via Vite) — core UI library and build tooling
- React Router v7 — client-side routing
- Axios — REST API communication
- Recharts — dashboards and analytics visualization
- Framer Motion — animation and page transitions

2. Frontend Technology Stack

Technology	Version	Purpose
Framework	React 19.2.7	Core UI library
Build Tool	Vite 8.1.1	Dev server and production bundler
Language	JavaScript (JSX)	No TypeScript in use
Routing	React Router DOM 7.18.1	Client-side routing
HTTP Client	Axios 1.19.0	REST API communication
State Management	React local state + 1 custom hook + 1 Context	No Redux / Zustand / Recoil present
Styling	Plain CSS + CSS custom properties	No Tailwind, CSS-in-JS, or component library
Charts / Visualization	Recharts 3.10.1	Analytics dashboards
Animation	Framer Motion 12.42.2	Page / element transitions
Icons	Lucide React 1.35.0, React Icons 5.7.0	Two icon libraries used across different components
Linting	ESLint 10.6.0	Code quality (npm run lint)

3. Frontend Architecture

The frontend follows a flat, page-centric architecture rather than a deeply layered one. Most business logic and API orchestration lives directly inside page components (in `src/pages/`), rather than being abstracted into dedicated state-management or controller layers.

```

User
|
v
Pages (role-specific dashboards: Beneficiary / Officer / Finance / Admin)
|
v
Reusable Components (AdminLayout, DashboardTopbar, ProfilePanel)
|
v
Local State (useState / useEffect) + useDashboardAnalytics hook
|
v
Services (Axios-based API modules: authService, schemeService, applicationService, etc.)
|
v
Shared Axios Client (api.js) -- with response interceptor
|
v
Backend REST API (Spring Boot)

```

Layer Responsibilities

- **Pages:** Own most of the page's logic — fetching data on mount, handling form submission, managing local UI state (loading/error/modals). This project does not separate "container" and "presentational" components; pages do both.
- **Reusable Components:** A small, deliberately shared set — AdminLayout (sidebar + header shell for the admin portal), DashboardTopbar (shared header across Officer/Finance/Admin dashboards), ProfilePanel (shared profile view/edit form used across all four roles).
- **Local State / Hooks:** No global state container. `useDashboardAnalytics` is the only custom hook, encapsulating a multi-endpoint dashboard data fetch for the Admin analytics view.
- **Services:** Thin wrapper functions around Axios calls, grouped by domain (`authService.js`, `schemeService.js`, `applicationService.js`, `adminService.js`, `officerService.js`, `fundsService.js`, `eligibilityService.js`). Some services — notably `schemeService.js` — also contain client-side data-shaping and a client-side eligibility rule evaluator.
- **Shared Axios Client (`api.js`):** Single instance with `withCredentials: true` (for the auth cookie) and a response interceptor that handles global 401 redirect-to-login logic.
- **Backend REST API:** Spring Boot, reached at a hardcoded `http://localhost:8080` base URL.

4. Frontend Folder Structure

```

frontend/
|-- public/
|   |-- eslint.config.js
|   |-- favicon.svg
|   |-- icons.svg
|   `-- logo.png
|-- src/
|   |-- assets/
|   |   |-- icons/           (favicon, icons, logo -- duplicated from public/)
|   |   |-- hero.png
|   |   |-- react.svg
|   |   `-- vite.svg
|   |-- components/         (Shared, reusable UI components -- 3 total)
|   |   |-- AdminLayout.jsx
|   |   |-- DashboardTopbar.jsx
|   |   `-- ProfilePanel.jsx
|   |-- context/
|   |   `-- ThemeContext.jsx (present but non-functional -- see Sec. 9)
|   |-- hooks/
|   |   `-- useDashboardAnalytics.js
|   |-- pages/
|   |   |-- Landing.jsx
|   |   |-- SchemeDetail.jsx
|   |   |-- scheme-dashboard.jsx
|   |   |-- admins/
|   |   |   `-- AdminDashboard.jsx
|   |   |-- auth/
|   |   |   |-- Login.jsx
|   |   |   `-- Register.jsx
|   |   |-- beneficiary/
|   |   |   |-- ApplicationTracking.jsx
|   |   |   |-- Dashboard.jsx
|   |   |   `-- FundsTracker.jsx
|   |   `-- officers/
|   |   |   |-- FinanceDashboard.jsx
|   |   |   |-- OfficerDashboard.jsx
|   |   |   `-- OfficerRegister.jsx
|   |-- services/           (Axios-based API modules, grouped by domain)
|   |   |-- adminService.js
|   |   |-- api.js          (shared Axios instance + interceptor)
|   |   |-- applicationService.js
|   |   |-- authService.js
|   |   |-- eligibilityService.js
|   |   |-- fundsService.js
|   |   |-- officerService.js
|   |   |-- schemeService.js
|   |   `-- sessionCleanup.js (clears localStorage/sessionStorage on logout/401)
|   |-- styles/             (per-page CSS files, not co-located with components)
|   |   |-- AdminLayout.css
|   |   |-- Dashboard.css
|   |   |-- FundsTracker.css
|   |   |-- Landing.css
|   |   |-- Login.css
|   |   `-- SchemeDetail.css
|-- App.jsx                 (root component -- router + route definitions)
|-- App.css                 (global styles -- separate token set from index.css)
|-- index.css               (global styles -- separate token set from App.css)
|-- main.jsx                (React root render entry point)

```

```
|-- index.html
|-- package.json
|-- vite.config.js
|-- vercel.json          (SPA rewrite rule for deployment)
```

Notable Structural Observations

- App.jsx and main.jsx sit at the frontend/ root, not inside src/ — the routing/entry point is not under src/pages or src/.
- Two separate global CSS files (App.css and index.css) each define their own :root design-token set with different color values — a real inconsistency, called out precisely in Section 13 (Styling Architecture).
- assets/icons/ and public/ both contain copies of the logo, favicon, and icons — duplication, not a build-generated artifact.

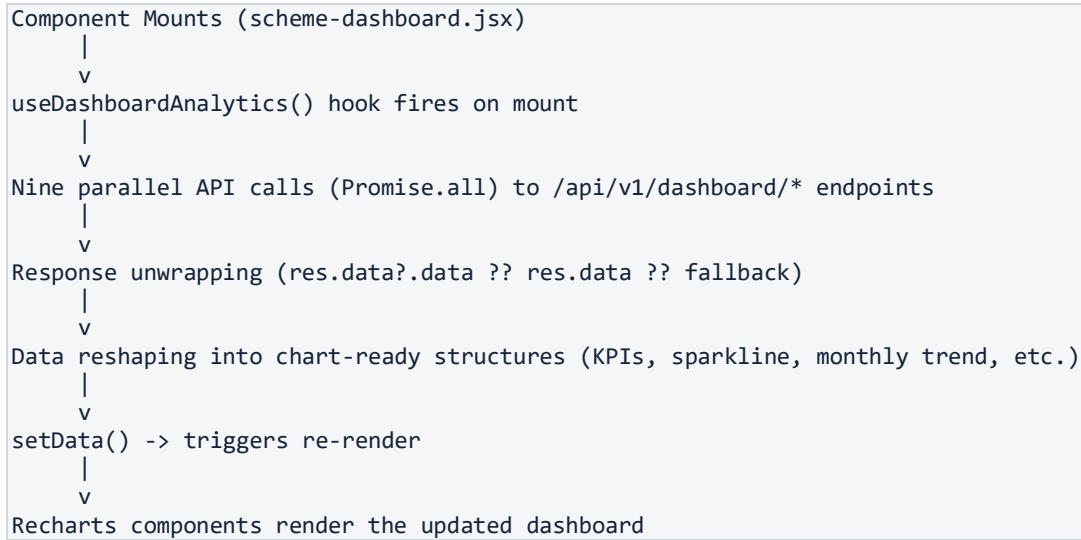
5. Application Flow

A typical user action in this application follows this path — using the login flow as a concrete, verified example:

```
User Action (submits login form)
  |
  v
UI Component (Login.jsx -- controlled form inputs via useState)
  |
  v
Event Handler (handleSubmit -- manual validation, e.g. non-empty checks)
  |
  v
Service Call (authService.login -> POST /gov/auth/signin via api.js)
  |
  v
Backend Endpoint (AuthController -> sets HttpOnly "token" cookie)
  |
  v
Response ({ status, message })
  |
  v
Follow-up API Call (GET /gov/auth/profile/get -- to determine the
                   user's role, since the login response itself
                   does not include it)
  |
  v
Local State Update (role extracted from profile response)
  |
  v
Navigation (role-based redirect: ADMIN -> /admin/dashboard,
           FINANCE_OFFICER -> /finance, *OFFICER -> /officer/dashboard,
           else -> /dashboard)
```

This pattern — page-owned state, a direct service call, then a manual navigate() — repeats across the application. There is no centralized routing guard or global auth state; each page independently calls GET /gov/auth/profile/get when it needs to know who is logged in (confirmed in Dashboard.jsx, AdminDashboard.jsx, OfficerDashboard.jsx, FinanceDashboard.jsx, ApplicationTracking.jsx, FundsTracker.jsx, and SchemeDetail.jsx).

Second Representative Flow — Dashboard Data Loading (Admin Analytics)



6. Routing

Routing library: React Router DOM v7 (BrowserRouter, Routes, Route), defined entirely in App.jsx (project root, not under src/).

Route Table (all 15 routes, verified from App.jsx)

Route	Component / Page	Access	Purpose
/	Landing	Public	Marketing / landing page
/login	Login	Public	Unified login (all roles)
/register	Register	Public	Beneficiary/Officer registration (tab-switchable)
/dashboard	Dashboard (beneficiary)	Beneficiary	Beneficiary home dashboard
/tracking/:schemeCode	ApplicationTracking	Beneficiary	Track application by scheme code
/funds/:schemeCode	FundsTracker	Beneficiary	View disbursement/milestone progress
/scheme/:id	SchemeDetail	Public/Authenticated	View scheme, apply, check eligibility
/officer/login	Login	Public	Same component as /login
/officer/register	Register	Public	Same component, defaults to officer mode
/officer/dashboard	OfficerDashboard	Officer	Officer's application review dashboard
/admin/login	(redirect)	-	Redirects to /login (<Navigate>)
/admin/dashboard	AdminDashboard	Admin	Admin dashboard
/admin	AdminDashboard	Admin	Alias for /admin/dashboard

Route	Component / Page	Access	Purpose
/finance	FinanceDashboard	Finance Officer	Finance officer's disbursement dashboard
/analytics	SchemeDashboard	Admin	Standalone analytics view (also embedded in OfficerDashboard)

Route Parameters

- `:schemeCode` — used in `/tracking/:schemeCode` and `/funds/:schemeCode`
- `:id` — used in `/scheme/:id`

Navigation Flow Notes

- `/admin` and `/admin/dashboard` both render `AdminDashboard` — duplicate routes to the same component.
- `scheme-dashboard.jsx` is both a standalone route (`/analytics`) and imported directly inside `OfficerDashboard.jsx` — it functions as both a page and a reusable component depending on context.
- Programmatic navigation (`useNavigate()`) is used throughout for post-action redirects (login, registration success, logout), rather than declarative `<Link>` navigation alone.

7. Components

Only three components are shared/reusable across multiple pages. All other UI is defined inline within individual page files — a notable architectural characteristic (see Section 20, Maintenance Guide).

7.1 AdminLayout

- Location: `src/components/AdminLayout.jsx`
- Purpose: Sidebar + header shell that wraps the Admin dashboard's content area.
- Props: `children`, `activeTab`, `onTabChange`, `userName`, `userRole`, `onLogout`
- State: None (fully controlled by parent props); reads `useLocation()/useNavigate()` internally for active-tab highlighting and route-based navigation.
- Important functions: `handleNavigation` (routes to a URL or calls `onTabChange`), `getInitials` (derives avatar initials from `userName`), `isItemActive` (determines active nav-item highlighting for both route-based and tab-based items).
- Dependencies: `lucide-react` icons, `react-router-dom`.
- Interacts with: Used exclusively by `AdminDashboard.jsx`.

7.2 DashboardTopbar

- Location: `src/components/DashboardTopbar.jsx`
- Purpose: Shared header bar (logo, user badge, home link, logout button) used across Officer, Finance, and Admin-adjacent dashboards.

- Props: brandTitle, brandSubtitle, userName, userRole, onLogout, homeLink, homeLabel, extraActions, showHomeLink — all with sensible defaults, making it configurable per-portal.
- State: None (fully controlled).
- Dependencies: framer-motion (button hover/tap animation), react-icons/fa.
- Interacts with: Used by OfficerDashboard.jsx, FinanceDashboard.jsx, and scheme-dashboard.jsx.

7.3 ProfilePanel

- Location: src/components/ProfilePanel.jsx
- Purpose: Shared profile view/edit form, reused across all four role dashboards with role-specific labeling.
- Props: profile, role, title, subtitle, editable, deletable, onSave, onDelete, fields (defaults to a standard 7-field set), plus label overrides (editLabel, saveLabel, cancelLabel, deleteLabel).
- State: draft (local edit buffer), isEditing, isSaving, error — all local useState.
- Important functions: buildDraft (initializes edit state from the profile prop), handleChange (with special digit-only masking for mobileNo), handleSubmit (calls the parent-supplied onSave and surfaces its errors locally).
- Dependencies: react-icons/fa.
- Interacts with: Used by Dashboard.jsx (beneficiary), AdminDashboard.jsx, and Officer/Finance dashboards for profile display.

8. Pages / Screens

8.1 Landing (Landing.jsx)

- Purpose: Public marketing/entry page.
- API calls: None identified -- this page does not call the backend.
- State: Local only (useState, useEffect, useMemo, useRef -- 8 hook usages), consistent with scroll-driven animation (useScroll, useTransform from Framer Motion are imported).
- Navigation: Links to /login and /register.

8.2 Login (auth/Login.jsx)

- Purpose: Unified login for all roles, plus a "forgot password" informational modal.
- UI structure: Split-panel layout -- left branding panel, right form panel.
- User interactions: Username/password entry, password visibility toggle, "Remember me" checkbox (present in UI, not wired to persistence logic), forgot-password modal (static contact-admin message, no reset flow).
- API calls: authService.login() -> POST /gov/auth/signin; on success, a follow-up GET /gov/auth/profile/get to determine role.
- State: form, showPw, error, loading, showForgotModal (all local useState).
- Navigation behavior: Role-based redirect after login (see Section 5).

8.3 Register (auth/Register.jsx)

- Purpose: Dual-mode registration -- Beneficiary or Officer -- switchable via in-page tabs.
- UI structure: Same split-panel layout as Login; a tab switcher toggles between BeneficiaryForm and OfficerForm sub-components.
- User interactions: Full field entry (name, mobile, region, district, state, username, password, confirm password); Officer mode adds a role dropdown.
- Validation: Manual, hand-written (see Section 12) -- 10-digit mobile, password complexity regex, password match.
- API calls: authService.register() or authService.registerOfficer() -> both POST /gov/auth/signup with a role field distinguishing them.
- State: mode, form, error, loading, toast.
- Navigation behavior: On success, shows a toast, then redirects to /login after 1.8s.

8.4 Officer Register (officers/OfficerRegister.jsx)

- Purpose: A standalone, officer-only registration page (separate from the dual-mode Register.jsx).
- API calls: authService.registerOfficer().
- State: 6 local useState hooks (form fields, loading, error).
- Notable: Appears to duplicate functionality already present in Register.jsx's officer mode -- worth confirming with the team whether both remain in active use.

8.5 Scheme Detail (SchemeDetail.jsx)

- Purpose: View a single scheme's details, check eligibility, submit an application, and upload required documents.
- API calls: schemeService.getSchemes(), applicationService.getApplications(), applicationService.submitApplicationBySchemeCode(), applicationService.cancelApplicationById(), applicationService.uploadApplicationDocuments(), eligibilityService.runEligibilityEngine(), plus a direct api.get('/gov/auth/profile/get') call.
- State: 22 local hook usages -- the most stateful page after the two largest dashboards, consistent with multi-step application submission and document upload in one page.
- Route: /scheme/:id

8.6 Beneficiary Dashboard (beneficiary/Dashboard.jsx)

- Purpose: Beneficiary's home dashboard -- browse schemes, view applications, manage profile.
- API calls: schemeService.getSchemes(), applicationService.getApplications(), fundsService.getCurrentBeneficiaryRecord(), plus direct api calls for profile get/update/delete and sign-out.
- Components used: ProfilePanel.
- State management: Local only.

8.7 Application Tracking (beneficiary/ApplicationTracking.jsx)

- Purpose: Track the status of a specific application by scheme code.
- API calls: `schemeService.getSchemes()`, `applicationService.getApplications()`, direct `profile-get` call.
- Route: `/tracking/:schemeCode`

8.8 Funds Tracker (beneficiary/FundsTracker.jsx)

- Purpose: View milestone-based disbursement progress for a beneficiary's application.
- API calls: `schemeService.getSchemes()`, `applicationService.getApplications()`, `fundsService.getCurrentBeneficiaryRecord()`, `fundsService.getDisbursementPlanByApplicationId()`, plus a direct `/api/media/upload` call (milestone proof evidence).
- Route: `/funds/:schemeCode`

8.9 Officer Dashboard (officers/OfficerDashboard.jsx)

- Purpose: Officer's main workspace -- review applications, run inspections, view embedded analytics.
- UI structure: Uses `DashboardTopbar` and `ProfilePanel`; embeds `SchemeDashboard` (`scheme-dashboard.jsx`) directly as a component for in-page analytics.
- API calls: Direct `profile-get/sign-out` calls confirmed; this is the single largest page in the codebase (1,814 lines). Its officer-specific service calls (from `officerService.js` -- inspection, milestone actions, allocation) are assumed present given that service's scope, though not every call was individually traced line-by-line.
- State: Local only.

8.10 Finance Dashboard (officers/FinanceDashboard.jsx)

- Purpose: Finance officer's workspace -- configure disbursement plans, review milestone proofs, release funds.
- UI structure: Uses `DashboardTopbar` and `ProfilePanel`.
- API calls: `officerService.getMyApplications()`, `officerService.getInspectionContext()`, plus direct `profile-get/sign-out` calls.
- State: Local only.

8.11 Admin Dashboard (admins/AdminDashboard.jsx)

- Purpose: The largest and most feature-dense page (1,753 lines) -- scheme management, officer approval, application allocation, audit log viewing, profile management.
- UI structure: Uses `AdminLayout` as its shell and `ProfilePanel` for profile management.
- API calls: `schemeService.getSchemes/addScheme/updateScheme`, `adminService.updateApprovalStatus/getOfficerRequests/getAllocationSummary/getProfilesByRole`,

applicationService.getApplications/allocateApplication/getAvailableOfficersWorkload/batchAllocateApplications, plus direct calls to /api/v1/disbursement/audit-logs, profile-get, and sign-out.

- State: Local only -- this page alone integrates five different service modules, making it the most cross-cutting page in the application.

8.12 Scheme Dashboard (scheme-dashboard.jsx)

- Purpose: Standalone analytics dashboard (also embedded inside OfficerDashboard).
- API calls: All via the useDashboardAnalytics() custom hook (9 parallel /api/v1/dashboard/* calls), plus adminService.getProfilesByRole() directly.
- State management: Delegates most data-state to the custom hook; local useState/useRef for UI-only concerns (animated counters via useMotionValue/animate/useInView).
- Route: /analytics; also usable as an embedded component.

9. State Management

Local State

The application relies entirely on React's built-in useState/useEffect hooks at the component level. Every page manages its own loading, error, form, and fetched-data state independently -- there is no shared or lifted state between pages.

Two mechanisms exist that approximate global concerns, both narrow in scope:

- ThemeContext (src/context/ThemeContext.jsx): A React Context wrapping the entire app via ThemeProvider in App.jsx. However, it is non-functional as a theme switcher -- theme is hardcoded to 'light', toggleTheme is a no-op function, and the useEffect unconditionally sets data-theme="light" on the document root regardless of any state change. It provides the shape of a theming system without an actual dark-mode implementation.
- sessionCleanup.js: Not state management per se, but a shared side-effect utility (clearPortalSessionCaches()) that clears a fixed list of localStorage/sessionStorage keys on logout or session-expiry, called from multiple pages and from the api.js interceptor.

Custom Hooks

One custom hook exists -- useDashboardAnalytics (src/hooks/useDashboardAnalytics.js). It encapsulates a Promise.all of nine dashboard API calls, response unwrapping, and data reshaping into chart-ready arrays, returning { data, loading, error } to the consuming component (scheme-dashboard.jsx).

Data Flow

Strictly top-down and page-local -- a page fetches its own data on mount via useEffect, stores it in useState, and passes it down to child components (ProfilePanel, AdminLayout, DashboardTopbar) as props. There is no prop-drilling of significant depth, since the shared components are shallow (page to component, not page to component to sub-component).

Persistence Mechanisms

Session/auth state is not persisted client-side beyond the browser's own handling of the HttpOnly cookie (see Section 11). sessionCleanup.js references four localStorage/sessionStorage cache keys (gov-subsidy-applications, gov-subsidy-officer-applications, gov-subsidy-finance-audit-logs, gov-subsidy-auth)

that are cleared on logout, implying some pages may cache fetched data under these keys for the session.

10. API Integration

API Client

A single shared Axios instance, defined in `src/services/api.js`.

Base URL

Hardcoded to `http://localhost:8080` (a commented-out production URL for a Render.com deployment is present but inactive). No `VITE_API_BASE_URL` environment variable is currently wired up, despite the README recommending one -- a discrepancy between documentation and implementation worth flagging to the team.

```
const api = axios.create({
  baseURL: 'http://localhost:8080',
  withCredentials: true,
  headers: {
    'Content-Type': 'application/json',
    'X-Requested-With': 'XMLHttpRequest',
  },
})
```

HTTP Methods Used

GET, POST, PUT, PATCH, DELETE -- all standard REST verbs, used consistently with their semantic meaning across services (e.g. PATCH for partial updates like approval status or scheme edits, DELETE for account/application removal).

Request Structure

Most POST/PUT/PATCH calls send a plain JSON object matching the shape expected by the backend's request DTOs (e.g. `{ username, password }` for login). File uploads (`uploadApplicationDocuments`, `uploadInspectionMedia`) construct a `FormData` object and override the `Content-Type` header to `multipart/form-data` for that specific call.

Response Structure

Backend responses are inconsistently shaped across endpoints -- some return the payload directly, others wrap it in a `{ data: ... }` envelope. Frontend services defensively unwrap with a repeated pattern:

```
return response.data?.data || response.data || <fallback>
```

This pattern appears in `adminService.js`, `fundsService.js`, `officerService.js`, and `schemeService.js`. It is a workaround for backend response-shape inconsistency rather than a deliberate frontend design choice.

Representative Endpoint Examples

Service Function	Method	Endpoint
authService.login()	POST	/gov/auth/signin
applicationService.submitApplicationBySchemeCode()	POST	/gov/applications/submit/{schemeCode}
adminService.bulkAllocateApplications()	POST	/api/v1/allocation/batch
fundsService.releaseMilestone()	POST	/api/v1/disbursement/release/{milestoneId}
officerService.submitInspectionReport()	POST	/api/officer/inspections/submit
schemeService.getSchemes()	GET	/gov/schemes/get (or /gov/schemes/get-{category})

Error Handling

Centralized in the api.js response interceptor:

- Extracts a human-readable message from `error.response?.data?.message`, falling back through `error.response?.data`, `error.message`, to a generic string.
- Always rejects with a plain `Error(message)`, so calling code can use `err.message` uniformly regardless of the original error shape.

Authentication Headers / Tokens

None sent explicitly by the frontend -- no `Authorization: Bearer <token>` header is constructed anywhere in the codebase. Authentication rides entirely on the `HttpOnly` cookie sent automatically by the browser (`withCredentials: true`).

Interceptors

One response interceptor, handling both error-message normalization and global 401 handling (see Section 11).

11. Authentication & Authorization

Login Flow

User submits credentials via `Login.jsx` -> `authService.login()` -> `POST /gov/auth/signin`. On success, the backend sets an `HttpOnly` token cookie. The frontend then makes a separate `GET /gov/auth/profile/get` call to retrieve the user's role, since the sign-in response does not include it, and redirects based on that role.

Registration Flow

Two entry points -- `Register.jsx` (dual-mode tab switcher for Beneficiary/Officer) and a standalone `OfficerRegister.jsx`. Both call `POST /gov/auth/signup` with a role field. Officer accounts require a separate Admin approval step (via `adminService.updateApprovalStatus()`) before they can log in -- stated explicitly in the Register UI copy and reflected in a dedicated officer-approval endpoint and Admin Dashboard section.

Token / Session Handling

JWT is issued and validated entirely server-side; the frontend has no code path that reads, stores, or manually attaches the token. This is a more secure pattern than typical localStorage-JWT setups, but it does mean the frontend cannot proactively detect an expired session before making a request -- it only learns of expiry when a request returns 401.

Storage Mechanism

No client-side token storage. sessionCleanup.js manages cache data (not the token itself) in localStorage/sessionStorage, cleared on logout or session-expiry.

Logout

Each dashboard page independently calls POST /gov/auth/signout directly via the shared api instance (not via a dedicated authService function), then calls clearPortalSessionCaches() and navigates to /login or /.

Role-Based UI Behavior

Roles (ADMIN, FINANCE_OFFICER, FIELD_OFFICER, DISTRICT_OFFICER, REGIONAL_OFFICER, BENEFICIARY) determine which dashboard a user lands on post-login, and ProfilePanel's ROLE_LABELS mapping adjusts display text per role. There is no single central "role guard" component -- role branching is done ad hoc per-page (most visibly in Login.jsx's redirect logic).

12. Forms & Validation

Forms Identified

Form	Location	Validation Approach
Login	auth/Login.jsx	Inline checks: non-empty username, non-empty password
Register	auth/Register.jsx	validate(form, mode): required fields, 10-digit mobile regex, password complexity regex, password match
Officer Register	officers/OfficerRegister.jsx	Same pattern as Register.jsx (separate implementation, not shared code)
Profile edit	ProfilePanel.jsx	Field-level masking only (digits-only, max 10 chars for mobileNo)
Application submission / upload	SchemeDetail.jsx	FormData construction for file uploads; not fully traced at line level

Validation Rules (Concrete, Verified)

- Mobile number: exactly 10 digits, non-digit characters stripped as the user types.
- Password (Register only): must start with an uppercase letter and include at least one lowercase letter, one digit, and one special character, minimum 8 characters total.
- Password confirmation: exact string match against the password field.
- All other required-field checks are simple .trim() emptiness checks.

Error Handling

Validation errors are stored in a local error state string and rendered inline above the submit button (with a framer-motion height/opacity transition in Login.jsx and Register.jsx). Server-side errors (e.g. "username not found," "incorrect password") are pattern-matched from the thrown error's message text in Login.jsx's catch block -- a fragile approach, since it depends on exact backend error-string wording, but this is what's implemented today.

Submission Flow

Standard onSubmit handler -> e.preventDefault() -> client-side validate() -> if valid, setLoading(true) -> API call -> success/error branch -> setLoading(false). No optimistic UI updates; all forms wait for the API response before changing view state.

13. UI / Styling Architecture

CSS Approach

Plain CSS files (no CSS-in-JS, no Tailwind, no CSS Modules) using CSS custom properties (design tokens) defined in :root. Each major page/feature area has its own dedicated stylesheet under src/styles/.

Theme System

A ThemeContext exists structurally but is non-functional -- hardcoded to a single light theme with a no-op toggle (see Section 9). There is no working dark mode despite the scaffolding suggesting one was planned.

A Real Inconsistency Worth Flagging

Two separate global stylesheets define two different, non-overlapping design-token systems:

- index.css -- a light, government-portal palette (--brand-green: #087443, white backgrounds, Inter/Source Serif 4 fonts).
- App.css -- a dark palette (--bg: #07130f, near-black backgrounds, amber/gold accents, Noto Sans/Source Serif 4 fonts).

Both are imported globally (index.css in main.jsx, App.css in App.jsx), meaning both token sets are active simultaneously in the browser at all times. Individual page stylesheets reference variables from one or the other set depending on which page they style -- so which "theme" a given page visually follows depends on which token set its CSS file happens to reference, not on any runtime theme logic.

Reusable Styles

The :root custom-property tokens (colors, spacing, shadows) serve as the de facto reusable style layer, referenced by value (var(--brand-green), var(--secondary), etc.) throughout component style={} props and CSS files alike.

Icons

Two separate icon libraries in simultaneous use -- lucide-react (used in AdminLayout.jsx) and react-icons/fa (Font Awesome, used in DashboardTopbar.jsx, ProfilePanel.jsx, Login.jsx, Register.jsx). No single icon system was standardized on.

Charts / Visualization

Recharts, used exclusively within useDashboardAnalytics-powered views (scheme-dashboard.jsx).

14. Environment Configuration

API URLs

Hardcoded in src/services/api.js (http://localhost:8080), with a commented-out alternate production URL (a Render.com deployment) left inactive in the same file.

External Services

None called directly from the frontend -- Cloudinary and other external services are backend-only integrations; the frontend only ever talks to its own backend, which then talks to Cloudinary server-side.

15. Frontend Setup

Prerequisites

- Node.js 18+ (per README; not independently version-pinned in package.json)
- A running instance of the backend (default http://localhost:8080) -- the frontend has no offline/mock mode

Installing Dependencies

```
cd frontend
npm install
```

Running Development Server

```
npm run dev
```

Starts Vite's dev server on http://localhost:5173 by default (per README; not overridden in vite.config.js, which contains no custom server.port setting).

Building Production Bundle

```
npm run build
```

Previewing Production Build

```
npm run preview
```

Other Available Script

```
npm run lint
```

All four commands (dev, build, lint, preview) are taken directly from the scripts block in package.json -- no additional or custom scripts exist beyond Vite's defaults.

16. Frontend Data Flow Examples

1. Login -> Role-Based Redirect

```

User submits login form (Login.jsx)
  |
  v
Manual validation (non-empty checks)
  |
  v
authService.login() -> POST /gov/auth/signin
  |
  v
Backend sets HttpOnly cookie, returns { status, message }
  |
  v
Follow-up: GET /gov/auth/profile/get (role not in login response)
  |
  v
Role branch -> navigate() to role-specific dashboard

```

2. Scheme Application Submission (SchemeDetail.jsx)

```

User views scheme detail page (/scheme/:id)
  |
  v
schemeService.getSchemes() fetches scheme list, page filters by :id
  |
  v
eligibilityService.runEligibilityEngine() checks eligibility
  |
  v
User submits application -> applicationService.submitApplicationBySchemeCode()
  |
  v
User uploads required documents -> applicationService.uploadApplicationDocuments()
  (constructs FormData, sends as multipart/form-data)
  |
  v
Backend processes submission + documents
  |
  v
Local state updates to reflect new application status

```

3. Admin Analytics Dashboard Load (scheme-dashboard.jsx)

```

Component mounts
  |
  v
useDashboardAnalytics() hook fires
  |
  v
Promise.all of 9 parallel GET requests to /api/v1/dashboard/*
  |
  v
Each response unwrapped (res.data?.data ?? res.data ?? fallback)
  |
  v
Raw data reshaped into chart-ready arrays (KPIs, sparkline, trends, etc.)
  |
  v
setData() in hook -> { data, loading, error } returned to component

```

v
 Recharts components re-render with new data

4. Officer Field Inspection Submission

```

Officer opens inspection modal for an application
|
v
officerService.getInspectionContext(applicationId) -> pre-fills modal
|
v
Officer uploads evidence photo(s) -> officerService.uploadInspectionMedia()
(multipart/form-data -> POST /api/media/upload -> returns media ID)
|
v
Officer submits report -> officerService.submitInspectionReport()
(POST /api/officer/inspections/submit, includes evidenceMediaIds)
|
v
Backend records inspection, updates application/workflow state
  
```

17. Error Handling

API Failures

Centralized in the `api.js` response interceptor (see Section 10). All rejected promises carry a normalized Error with a human-readable `.message`, extracted from the backend's error response body where available. Individual pages catch this in their own `try/catch` blocks and display it via a local error state string.

Validation Failures

Handled entirely client-side before any API call is made, via the manual `validate()` functions described in Section 12. Errors are shown inline, not as toasts, except in `Register.jsx` which uses a toast only for the success case.

Unauthorized Requests (401)

Handled globally by the `api.js` interceptor -- clears session caches (`clearPortalSessionCaches()`) and redirects to `/login`, with a guard flag (`redirectingToLogin`) to prevent multiple simultaneous redirects if several requests fail at once. Explicitly bypassed for the two auth endpoints themselves (`/gov/auth/signin`, `/gov/auth/signup`) and when already on a login/register page, so a failed login attempt shows its own error message instead of forcing a redirect loop.

Loading States

Handled per-page via a local loading boolean, typically shown as a spinner or a disabled submit button during in-flight requests. `useDashboardAnalytics` exposes its own loading flag for the analytics dashboard.

Network Errors (Backend Unreachable)

`Login.jsx`'s catch block includes an explicit fallback message -- "Server is down or unavailable" -- for errors that don't match its known error-string patterns. No retry logic or offline detection exists elsewhere in the codebase.

18. Build & Deployment

Production Build

```
npm run build
```

Runs vite build, producing a static bundle (default output: dist/) -- no custom build configuration exists in vite.config.js beyond the React plugin.

Deployment Target

Vercel -- confirmed by the presence of frontend/vercel.json:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

This is a standard single-page-application rewrite rule, ensuring all paths resolve to index.html so React Router can take over client-side routing, rather than Vercel returning a 404 for unknown paths.

19. Frontend Maintenance Guide

Add a New Page

- Create the .jsx file under the appropriate src/pages/ subfolder (or a new one, following the existing admins/ / auth/ / beneficiary/ / officers/ grouping).
- Add a corresponding <Route> entry in App.jsx (project root).
- If the page needs a shared header/shell, use DashboardTopbar or AdminLayout rather than building a new one.

Add a New Component

- Only extract a new shared component if genuinely reused across 2+ pages -- the existing pattern favors page-local JSX over premature abstraction.
- Place it in src/components/.

Add a New API Endpoint Integration

- Add a new exported function to the relevant domain service file (e.g. a new disbursement-related call goes in fundsService.js), following the existing pattern of a thin async wrapper around api.<method>().
- If the backend response shape is inconsistent, apply the existing response.data?.data || response.data || <fallback> defensive-unwrap pattern for consistency.

Change Navigation

- Route paths are defined only in App.jsx -- this is the single place to add, remove, or rename routes.
- Redirect logic embedded in components (e.g. Login.jsx's role-based redirect) must be updated separately if role names or target routes change -- there is no central route-mapping config.

Change Styling

- Be aware of the two competing global token systems (index.css vs App.css -- see Section 13) before adding new var(--...) references; confirm which token set the target page's stylesheet actually uses.
- Per-page CSS lives in src/styles/, one file per page/feature area -- follow that convention rather than introducing co-located component CSS.

Add a New Form

- Follow the existing hand-rolled pattern: a form state object via useState, a dedicated validate() function returning an error string or null, and a local error state for display -- there is no validation library to plug into.

Add New Environment Configuration

- There is currently no .env infrastructure to extend. Introducing one would require adding Vite's import.meta.env.VITE_* convention from scratch, updating api.js to read from it instead of the hardcoded baseUrl, and creating a .env.example file.

20. Frontend Troubleshooting

"Network Error" / all API calls fail immediately

Likely cause: backend not running, or not running on http://localhost:8080. Since the base URL is hardcoded in api.js with no environment override, confirm the backend's actual port/host matches that hardcoded value, or edit api.js directly for local testing against a different backend location.

User gets redirected to /login unexpectedly, even when recently logged in

Likely cause: JWT cookie expired, or the backend-issued cookie's expiry is shorter than expected. Since there's no client-side session-expiry warning (see Section 11), this will always present as an abrupt redirect after a failed request, not a graceful warning beforehand.

Styling looks inconsistent between pages (mismatched color palette)

Expected, given the two separate global design-token systems described in Section 13. Confirm which stylesheet (index.css vs App.css token set) the affected page's CSS file references before assuming a bug.

Officer registration succeeds but the officer can't log in

Expected behavior, not a bug -- officer accounts require Admin approval (via the Admin Dashboard's officer-request review flow) before login is permitted, per the Register page's own UI copy and the existence of a dedicated approval endpoint.

File upload fails for large documents / inspection media

Check the backend's configured multipart limits (spring.servlet.multipart.max-file-size=40MB, max-request-size=100MB -- confirmed in backend application.properties) -- the frontend does not independently enforce or surface a smaller limit before attempting upload.

CORS errors in the browser console

Should not occur under default configuration -- the backend's SecurityConfig allows all origin patterns (*) with credentials enabled. If CORS errors do appear, check for a mismatch between the frontend's actual origin and any future tightening of the backend's CORS policy.

Build succeeds but routing breaks in production (404s on refresh)

Confirm vercel.json's rewrite rule is actually being honored by the deployment platform in use -- this rule is Vercel-specific; deploying to a different static host would require an equivalent rewrite/fallback rule to be configured there instead, since the SPA fallback isn't handled by Vite itself.